

Find the Order

- **Batas waktu:** 10 menit
- **Lingkungan:** satu GPU (≈ 16 GB VRAM), tanpa internet
- **Ukuran solusi:** `solution.ipynb` ≤ 1 MB
- **Penyimpanan:** 5 GB

Soal

Anda diberikan dialog lisan berbahasa Inggris antara dua peserta, *Speaker A* dan *Speaker B*. Setiap dialog disegmentasi menjadi giliran bicara (*turn*), dengan setiap giliran hanya berisi ucapan dari satu pembicara. Setiap giliran disimpan sebagai berkas audio `.wav` terpisah, sehingga satu dialog utuh direpresentasikan oleh sekumpulan berkas `.wav`, dimana satu berkas `.wav` untuk setiap giliran.

Sayangnya, giliran-giliran tersebut telah diacak secara acak (*random*), sehingga percakapannya tidak lagi masuk akal. Pada nama berkas `chunk_{k}.wav`, indeks k merujuk pada *chunk* ke- k yang berisi ucapan dari satu pembicara di dalam kumpulan ucapan yang telah diacak, bukan giliran ke- k dalam dialog aslinya.

!! Tugas Anda adalah merekonstruksi urutan kronologis asli dari percakapan tersebut.



Dataset

Setiap dialog berisi n berkas audio bernama `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. *Chunk* merupakan giliran bicara individual yang sudah diacak secara *random*. Nama berkas hanya berkaitan dengan urutan acak tersebut. Nama berkas tidak menunjukkan posisi suatu *chunk* dalam percakapan aslinya. Setiap dialog memiliki 7–20 *chunk*, *mono*, 44.1 kHz (Anda boleh melakukan resampling).

`prefix.json` berisi indeks nama berkas `.wav` dari dua *chunk* ucapan pertama pada setiap dialog. Hal ini menandai awal sebenarnya dari dialog dan menghilangkan ambiguitas urutan ucapan dari sebuah dialog/percakapan (misalkan, antara membaca percakapan maju atau mundur).

Sebagai contoh: Informasi 11: `[7, 12]` pada `prefix.json` menandakan bahwa giliran pertama dan kedua dari dialog/percakapan ke-11 adalah `chunk_7.wav` dan `chunk_12.wav`.

Apa yang Anda dapatkan

Anda menerima **dua folder dengan format yang identik:**

Nama Folder	Jumlah Dialog	Akses ke <code>answers.json</code> ?	Digunakan Untuk
<code>dataset/train/</code>	1,288	☐ disertakan	Melatih / melakukan <i>fine-tune</i> model Anda
<code>dataset/test_public/</code>	100	☐ disertakan	Menjalankan <i>pipeline</i> Anda dan menilai sendiri secara lokal

Pada saat penilaian, *path* ke folder `dataset/test_public/` yang Anda miliki akan secara otomatis digantikan oleh sebuah *path* ke hidden evaluation set (`test_leaderboard_a` untuk leaderboard publik dan `test_leaderboard_b` untuk leaderboard final) — folder-folder ini memiliki ukuran dan format yang sama dengan `dataset/test_public/` tetapi tanpa `answers.json`.

Notebook Anda dijalankan kembali pada data tersebut, dan berkas `answers.json` yang dihasilkannya digunakan untuk penilaian. Dialog uji yang disembunyikan pada hidden evaluation set berasal dari distribusi yang sama dengan `train`, sehingga skor `test_public` lokal Anda merupakan pratinjau yang akurat.

Struktur direktori

```
dataset/train/
  prefix.json  # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P}  ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json      # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Keluaran

Untuk setiap dialog, tentukan urutan kronologis asli dari *chunk* audionya. Prediksi Anda harus berupa permutasi P dari $\{0, 1, \dots, n-1\}$, dengan $P[i]$ adalah posisi kronologis prediksi dari *chunk_i.wav* (dimana indeks 0 = pertama).

Berkas keluaran Anda *answers.json* harus memetakan setiap ID dialog ke permutasi prediksinya:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Contoh

Sebuah dialog memiliki 3 chunk teracak *chunk_0*, *chunk_1*, *chunk_2*:

chunk teracak	isi ucapan	posisi sebenarnya (peringkat)
chunk_0.wav	"Tidak apa-apa — nanti saya kirimkan catatannya."	2 (terakhir)
chunk_1.wav	"Hei, apakah kamu datang ke rapat pukul tiga?"	0 (pertama)
chunk_2.wav	"Saya tidak bisa — saya ada janji dengan dokter gigi saat itu."	1

Urutan sebenarnya adalah **chunk_1** → **chunk_2** → **chunk_0**, sehingga $P = [2, 0, 1]$ (urutan permutasi untuk chunk_0.wav, chunk_1.wav, dan chunk_2.wav di dialog tersebut) dan prefix.json berisi $[1, 2]$ (untuk chunk_1.wav dan chunk_2.wav).

⚠ **P harus merupakan permutasi yang sah:** panjang n , berindeks mulai dari 0, setiap nilai muncul tepat satu kali. Nilai duplikat, nilai yang hilang, atau entri di luar rentang (misalnya berindeks mulai dari 1) mendapat skor 0 untuk dialog tersebut, demikian pula dialog yang tidak ada dalam berkas. Berkas yang rusak formatnya atau bukan JSON akan ditolak.

Penilaian

Penilaian untuk tugas ini adalah **akurasi pengurutan berpasangan (pairwise ordering accuracy)**. Penilaian ini memeriksa setiap pasangan chunk dan menanyakan: *yang mana dari keduanya yang seharusnya lebih dahulu?* Sebuah pasangan dianggap benar jika prediksi Anda memberikan jawaban yang sama dengan ground truth. Untuk sebuah dialog dengan n chunk terdapat

$$M = n(n - 1)/2$$

pasangan; misalkan I adalah banyaknya inversi — pasangan yang urutannya berbeda dari ground truth:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i **Skor akhir adalah rata-rata skor per dialog atas seluruh dialog dalam split tersebut.**

Model yang diizinkan

Anda hanya boleh menggunakan model pra-terlatih (*pre-trained*) berikut untuk menyelesaikan tugas ini, baik selama pelatihan maupun evaluasi. Semua model ini sudah diunduh dan tersedia di lingkungan kerja. Anda dapat melihat contoh cara menggunakannya pada notebook baseline solution.ipynb. Perlu diperhatikan bahwa Anda tidak boleh menggunakan model lain, dan program Anda tidak memiliki akses internet.

- **Representasi ucapan: wav2vec 2.0** → [wav2vec model card](#)
- **Pengenalan ucapan otomatis (ASR): Whisper encoder** juga boleh digunakan sebagai ekstraktor fitur → **OpenAI Whisper** (ukuran apa pun). [Whisper model card](#)
- **Model bahasa: Qwen2.5-0.5B**, yang boleh digunakan secara *zero-shot* atau di-*fine-tune* pada `split train` yang disediakan → [Qwen model card](#).

Perhatikan bahwa batas 10 menit harus mencakup seluruh pelatihan atau fine-tuning yang Anda lakukan pada saat penilaian ditambah inferensi pada set evaluasi.

Cara mengumpulkan

- Buka `solution.ipynb` dan jalankan semua sel. Pastikan notebook menulis `answers.json` di direktori kerja dengan sebuah permutasi untuk setiap dialog dalam `dataset/test_public/` (100 dialog). Pada saat penilaian, notebook dijalankan ulang pada set uji tersembunyi dan `answers.json` yang dihasilkannya di sana yang dinilai.
- Tingkatkan solusi jika Anda mau — atau tidak; baseline saja sudah memvalidasi pipeline.
- Buka tab Git di bilah sisi kiri JupyterLab.
- **Stage** `solution.ipynb` (ikon + di sebelahnya).
- Masukkan pesan commit dan klik **Commit**.
- Klik ikon awan dengan panah ke atas untuk melakukan push.
- Kembali ke halaman Contest ini dan klik **Submit**.

Kumpulkan tepat satu berkas, bernama `solution.ipynb`.